Multi-User AI Chatbot with Supabase JSONB Database

Student Notes - Persistent Conversation History with LangChain

Project Idea

Build a multi-user AI chatbot where each user can sign up, log in, chat with the AI, and continue the same conversation later. User profiles and chat messages are stored in Supabase PostgreSQL using JSONB columns.

What Students Will Learn

- How a Python application connects to Supabase using SUPABASE_URL and SUPABASE_SECRET_KEY.
- How JSONB can store flexible document-style data inside PostgreSQL.
- How signup and login data can be stored and read from a Supabase table.
- How RunnableWithMessageHistory connects each session to the correct conversation history.

- How LangChain messages are converted to dictionaries before saving and restored when reading.
- How the Supabase backend can be opened in Table Editor to show students the real stored data.

Very Important

Supabase is PostgreSQL, not a native NoSQL database. In this project we use JSONB for NoSQL-style / document-style storage.

Program Flow - Big Picture

Observe this diagram first. Then study the code section-by-section.

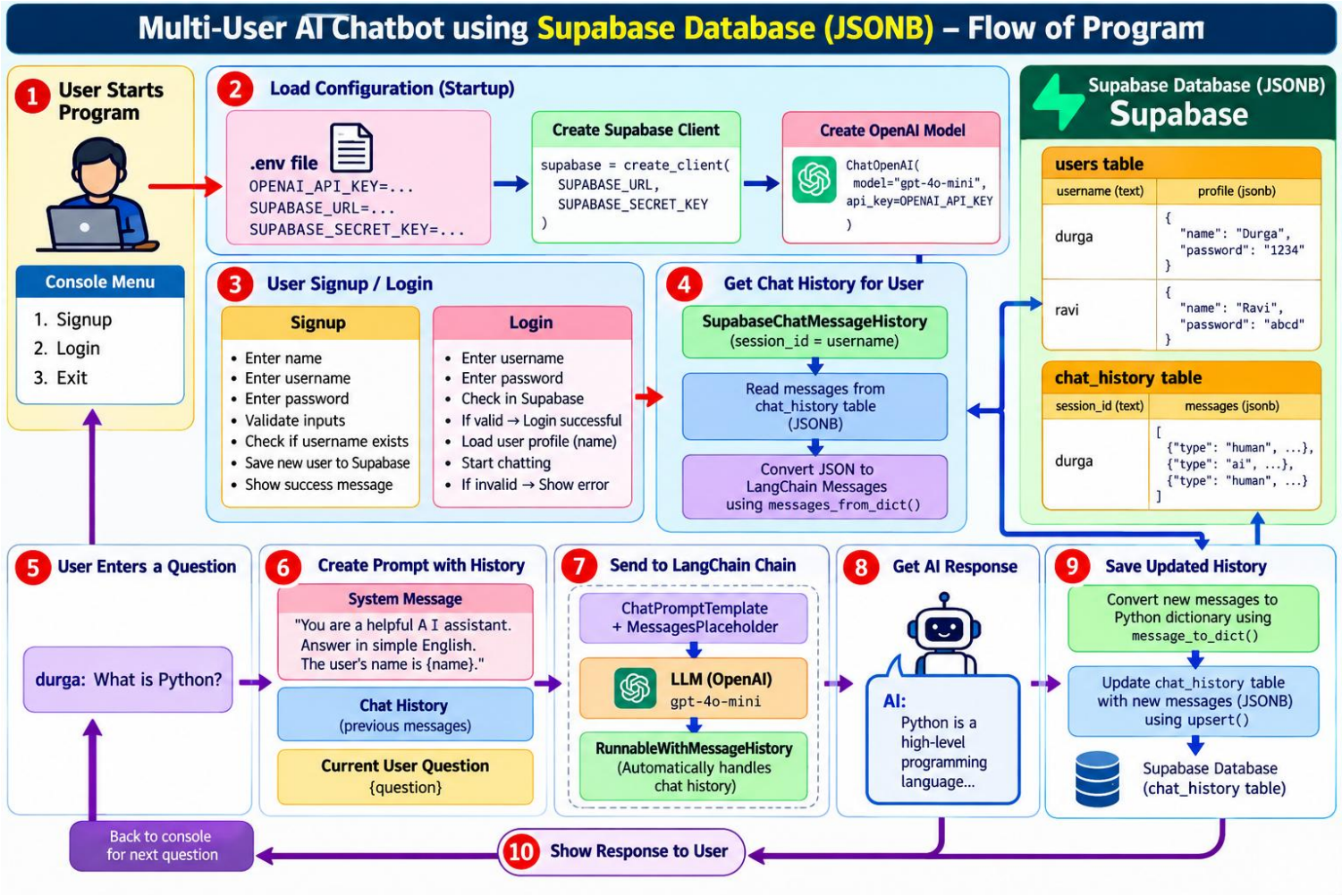
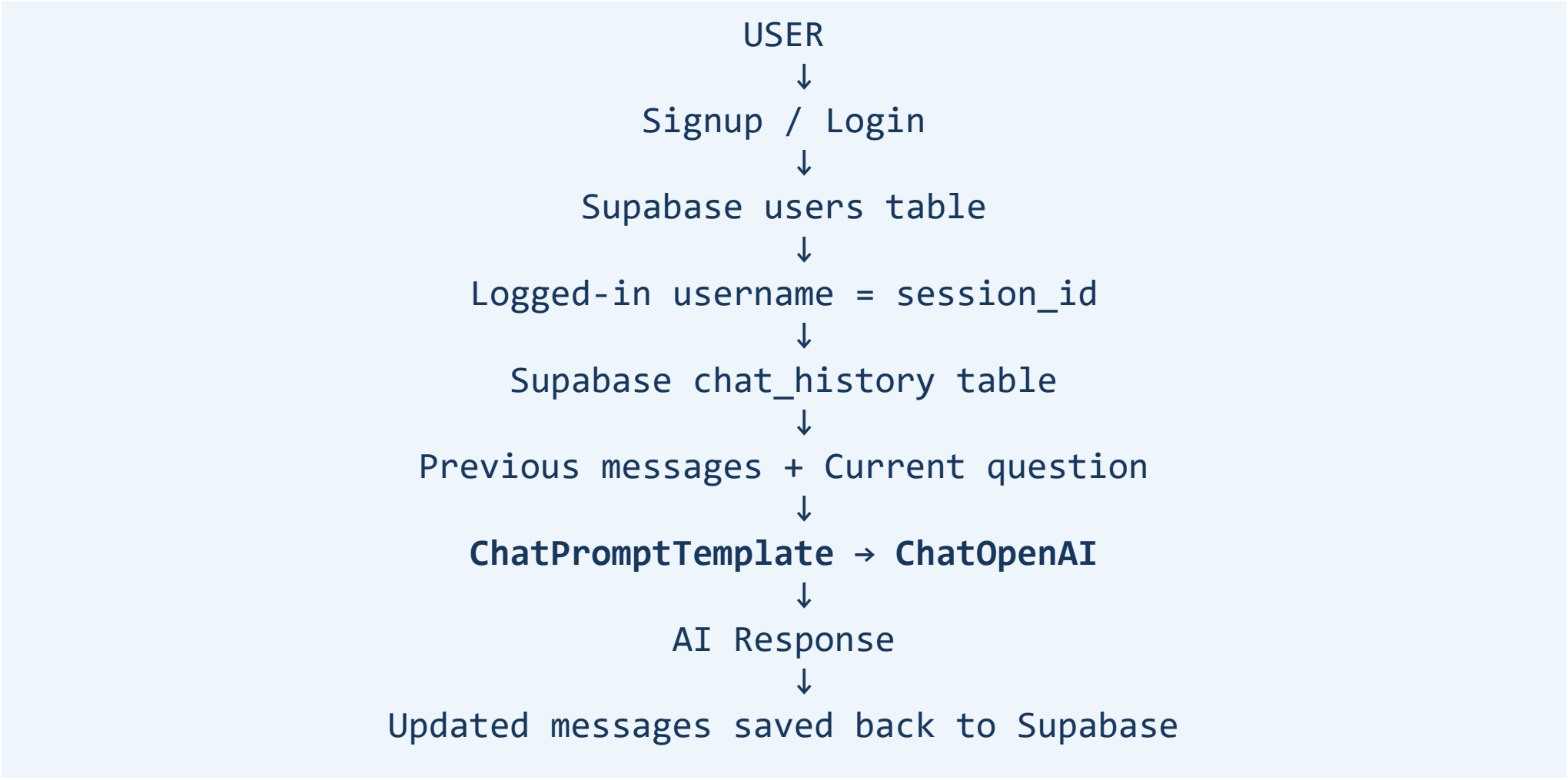


Figure: Complete Python program flow using Supabase JSONB and LangChain

1. Project Architecture



Two Tables Used in This Project

Table	Main Columns	Purpose
users	username, profile JSONB	Stores registered user information
chat_history	session_id, messages JSONB	Stores one persistent conversation per session/user

2. Create the Supabase Tables

Open Supabase Dashboard → SQL Editor → New Query. Paste and run:

```
create table if not exists public.users (
    username text primary key,
    profile jsonb not null
);

create table if not exists public.chat_history (
    session_id text primary key,
    messages jsonb not null default '[]'::jsonb
);
```

Backend View
After the program runs, open Supabase Dashboard → Table Editor. You can show students the users table and chat_history table directly.

Example Stored Data

```
users table
-----------
username: durga
profile:  {"name": "Durga", "password": "1234"}

chat_history table
------------------
session_id: durga
messages:   [ { ...HumanMessage... }, { ...AIMessage... } ]
```

3. Environment Configuration

Create a file named .env in the same project folder as the Python program:

```
OPENAI_API_KEY=YOUR_OPENAI_API_KEY
SUPABASE_URL=https://YOUR_PROJECT.supabase.co
SUPABASE_SECRET_KEY=YOUR_SUPABASE_SECRET_KEY
```

Security Rule

SUPABASE_SECRET_KEY is a backend secret. Never put it in browser JavaScript, public GitHub code, screenshots, or student handouts with the real value.

Install Required Packages

```
pip install supabase python-dotenv langchain-openai langchain-core
```

4. Step 1 - Connect Python to Supabase

```
load_dotenv()

supabase = create_client(
    os.environ["SUPABASE_URL"],
    os.environ["SUPABASE_SECRET_KEY"]
```

```
)

USERS_TABLE = "users"
HISTORY_TABLE = "chat_history"
```

```
                    .env
                     ↓
                load_dotenv()
                     ↓
        SUPABASE_URL + SUPABASE_SECRET_KEY
                     ↓
               create_client()
                     ↓
          Python ⇔ Supabase Connection
```

5. Step 2 - Load Registered Users

```
def load_users():

    response = (
```

```
        supabase
        .table(USERS_TABLE)
        .select("username, profile")
        .execute()
    )

    users = {}

    for row in response.data:
        users[row["username"]] = row["profile"]

    return users
```

Simple meaning:

```
                    users table
                        ↓
            SELECT username, profile
                        ↓
                   response.data
                        ↓
          Convert rows into Python dictionary
```

```
                    ↓
                  users
```

6. Step 3 - Save Users

```
def save_users(users):

    rows = []

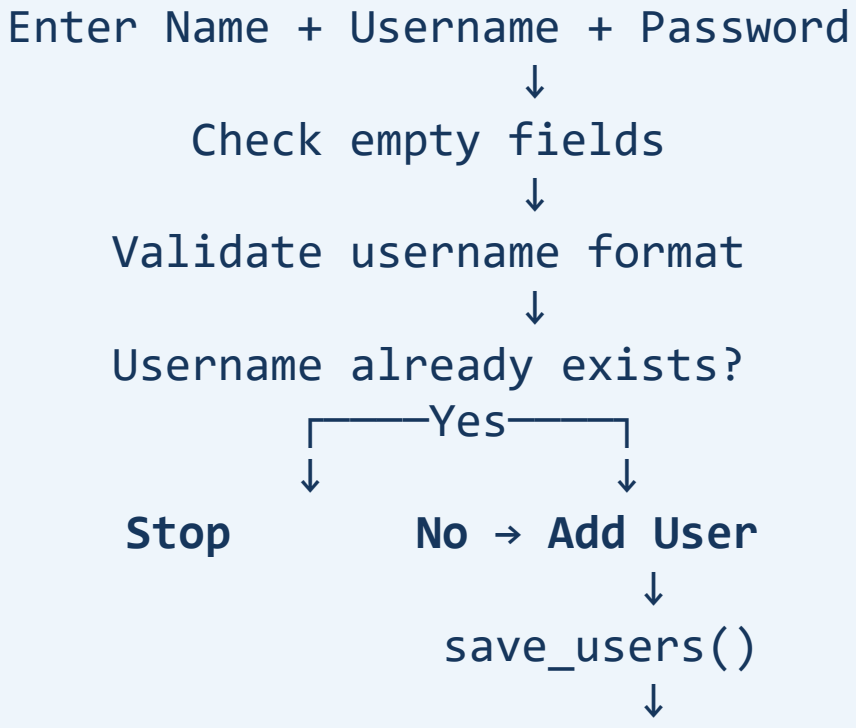
    for username, profile in users.items():
        rows.append({
            "username": username,
            "profile": profile
        })

    if rows:
        (
            supabase
            .table(USERS_TABLE)
            .upsert(rows, on_conflict="username")
            .execute()
```

```
        )
```

What is upsert?
If the row does not exist, INSERT it. If it already exists, UPDATE it.

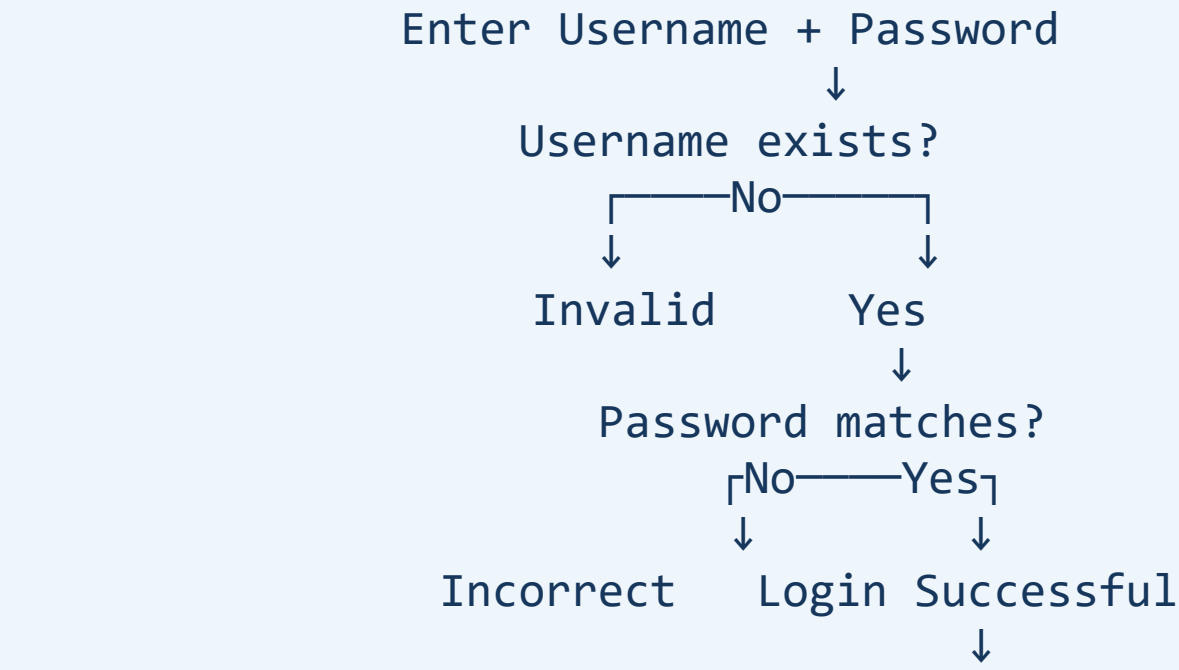
7. Step 4 - Signup Logic



```
Supabase
```

- The username is converted to lowercase so DURGA, Durga, and durga do not become three separate usernames.
- The regular expression r"[a-z0-9_]+" allows lowercase letters, numbers, and underscore.
- The user profile is stored in the JSONB profile column.

8. Step 5 - Login Logic



```
return username
```

Important Connection
The username returned by login becomes current_user. Later, current_user is also used as session_id.

9. Step 6 - SupabaseChatMessageHistory

This is the heart of persistent conversation history.

```
class SupabaseChatMessageHistory(
    BaseChatMessageHistory
):

    def __init__(self, session_id):
        self.session_id = session_id
```

```
session_id = "durga"
          ↓
```

```
SupabaseChatMessageHistory
        ↓
Find row where session_id = "durga"
        ↓
Use Durga's messages only
```

10. Reading Chat History

```
@property
def messages(self):

    response = (
        supabase
        .table(HISTORY_TABLE)
        .select("messages")
        .eq("session_id", self.session_id)
        .execute()
    )

    if not response.data:
        return []
```

```
    data = response.data[0]["messages"]

    if not data:
        return []

    return messages_from_dict(data)
```

```
                    session_id
                        ↓
    SELECT messages FROM chat_history
    WHERE session_id = current session
                        ↓
                JSONB dictionaries
                        ↓
            messages_from_dict()
                        ↓
        HumanMessage / AIMessage objects
```

11. Saving Chat History

```
def add_messages(self, messages):

    all_messages = self.messages + messages

    data = [
        message_to_dict(message)
        for message in all_messages
    ]

    (
        supabase
        .table(HISTORY_TABLE)
        .upsert(
            {
                "session_id": self.session_id,
                "messages": data
            },
            on_conflict="session_id"
        )
        .execute()
```

```
    )
```

```
                Old Messages
                      +
                New Messages
                      ↓
            message_to_dict()
                      ↓
        JSON-compatible dictionaries
                      ↓
          UPSERT chat_history row
                      ↓
        Persistent JSONB history
```

12. Serialization and Deserialization

Function	Conversion	Why Used
message_to_dict()	LangChain Message → Python Dictionary	Prepare message for JSONB storage

messages_from_dict()	Dictionaries → LangChain Messages	Restore messages for the prompt/model

SAVE SIDE
Message → message_to_dict() → JSONB

LOAD SIDE
JSONB → messages_from_dict() → Message

13. Step 7 - Create Model, Prompt and LCEL Chain

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.environ["OPENAI_API_KEY"]
)

prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "You are a helpful AI assistant. "
```

```
        "Answer in simple English. "
        "The user's name is {name}."
    ),
    MessagesPlaceholder("history"),
    ("human", "{question}")
])

chain = prompt | llm
```

```
System Message
      +
Previous History
      +
Current Question
      ↓
ChatPromptTemplate
      ↓
  ChatOpenAI
      ↓
  AI Response
```

14. Step 8 - RunnableWithMessageHistory

```
def get_session_history(session_id):
    return SupabaseChatMessageHistory(session_id)

chatbot = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="question",
    history_messages_key="history"
)
```

Item	Meaning
chain	Original Runnable: prompt | llm
get_session_history	Returns the correct Supabase history object
input_messages_key="question"	Current user message is in the question field
history_messages_key="history"	Previous messages are inserted into the history placeholder

15. Step 9 - Main Application Loop

```
START
  ↓
load_users()
  ↓
current_user = None
  ↓
Signup / Login / Exit Menu
  ↓
Login Successful
  ↓
current_user = username
  ↓
Chat Loop
  ↓
logout → Back to Menu
exit → End Program
```

16. Step 10 - One Chat Request

```
response = chatbot.invoke(
    {
        "name": name,
        "question": question
    },
    config={
        "configurable": {
            "session_id": current_user
        }
    }
)
```

Most Important Point

current_user is the username, and the same value is used as session_id. Therefore each user automatically gets separate persistent conversation history.

```
current_user = "durga"
                ↓
```

```
session_id = "durga"
        ↓
chat_history row for durga
        ↓
Durga History Only
```

17. How to Show the Backend to Students

- Open Supabase Dashboard.
- Open Table Editor.
- Open users table and show username + profile JSONB.
- Open chat_history table and show session_id + messages JSONB.
- Login as another user and chat.
- Refresh Table Editor and show that a separate chat_history row appears.
- Stop Python, start it again, login with the same user, and prove that old conversation history is still available.

Classroom Demonstration

```
Durga logs in
```

```
                    ↓
"My favourite language is Python"
                    ↓
            Saved in Supabase
                    ↓
          Stop Python Program
                    ↓
          Start Program Again
                    ↓
             Login as Durga
                    ↓
"What is my favourite language?"
                    ↓
Old history is loaded from Supabase
```

18. Important Concepts

Concept	Meaning in This Project
Persistent State	Data remains in Supabase even after Python stops

JSONB	Flexible JSON/document-style data stored in PostgreSQL
session_id	Identifies which conversation history should be used
current_user	Username of the logged-in user
Message History	Sequence of previous HumanMessage and AIMessage objects
RunnableWithMessageHistory	Wraps the chain and automatically reads/updates the selected history

19. Common Mistakes

- Do not say Supabase is a NoSQL database. It is PostgreSQL; JSONB gives us document-style storage.
- Do not use the publishable key as the backend secret key for this server-side demo.
- Do not expose SUPABASE_SECRET_KEY in frontend code or public repositories.
- Do not forget to create the users and chat_history tables before running the program.
- Do not change input_messages_key="question" unless the input dictionary key also changes.
- Do not change history_messages_key="history" unless the MessagesPlaceholder name also changes.

- Same model does not mean same history. session_id decides which history is selected.

Production Security Note

This classroom program stores passwords as plain text inside JSONB only to keep the learning flow simple. For a real application, use Supabase Auth or secure password hashing.

20. 15 Key Points for Revision

- Supabase is PostgreSQL; JSONB provides flexible document-style storage.
- The users table stores username and profile JSONB.
- The chat_history table stores session_id and messages JSONB.
- SUPABASE_URL and SUPABASE_SECRET_KEY are loaded from .env.
- load_users() reads user rows from Supabase.
- save_users() uses upsert to insert or update rows.
- Signup validates and saves a new user.
- Login returns the username when credentials match.
- The username becomes current_user after login.
- current_user is used as session_id.
- SupabaseChatMessageHistory loads and saves the selected session history.

- messages_from_dict() restores stored JSON data into LangChain messages.
- message_to_dict() converts LangChain messages into JSON-compatible dictionaries.
- RunnableWithMessageHistory connects the LCEL chain with persistent session history.
- The same user can restart the program and continue the old conversation.

21. Quick MCQs

1. Which Supabase data type stores flexible user/profile data in this project?

A. INTEGER
B. JSONB
C. DATE
D. BOOLEAN

Answer: B - JSONB stores flexible JSON/document-style data.

2. What identifies the correct conversation history?

A. Model name
B. Password
C. session_id
D. Temperature

Answer: C - session_id selects the correct conversation.

3. What does message_to_dict() do?

A. Message → Dictionary
B. Dictionary → Message
C. SQL → JSON
D. Deletes history

Answer: A - It prepares LangChain messages for JSONB storage.

4. What does upsert mean?

A. Only SELECT
B. Insert if missing or update if existing
C. Delete all rows
D. Create model

Answer: B - Upsert combines insert/update behaviour.

5. Why can history survive a Python restart?

A. The LLM remembers forever
B. It is stored in Supabase
C. Python list is permanent
D. The prompt saves it

Answer: B - Supabase is persistent external storage.

22. Complete Ready-to-Execute Program

Copy this program only after creating the two Supabase tables and configuring the .env file.

```
import os
import re

from dotenv import load_dotenv
from supabase import create_client

from langchain_openai import ChatOpenAI

from langchain_core.prompts import (
    ChatPromptTemplate,
    MessagesPlaceholder
)

from langchain_core.chat_history import (
    BaseChatMessageHistory
)

from langchain_core.messages import (
```

```
    message_to_dict,
    messages_from_dict
)

from langchain_core.runnables.history import (
    RunnableWithMessageHistory
)


# ========================================
# 1. SUPABASE CONFIGURATION
# ========================================

load_dotenv()

supabase = create_client(
    os.environ["SUPABASE_URL"],
    os.environ["SUPABASE_SECRET_KEY"]
)

USERS_TABLE = "users"
HISTORY_TABLE = "chat_history"
```

```
# ========================================
# 2. LOAD USERS
# ========================================

def load_users():

    response = (
        supabase
        .table(USERS_TABLE)
        .select("username, profile")
        .execute()
    )

    users = {}

    for row in response.data:
        users[row["username"]] = row["profile"]

    return users
```

```
# ========================================
# 3. SAVE USERS
# ========================================

def save_users(users):

    rows = []

    for username, profile in users.items():

        rows.append(
            {
                "username": username,
                "profile": profile
            }
        )

    if rows:

        (
            supabase
```

```
            .table(USERS_TABLE)
            .upsert(
                rows,
                on_conflict="username"
            )
            .execute()
        )


# ========================================
# 4. USER SIGNUP
# ========================================

def signup(users):

    print("\n--- SIGNUP ---")

    name = input("Enter Name: ").strip()

    username = input(
        "Enter Username: "
    ).strip().lower()
```

```
password = input("Enter Password: ")

if not name or not username or not password:
    print("All fields are required")
    return

if not re.fullmatch(r"[a-z0-9_]+", username):
    print("Use only letters, numbers and underscores")
    return

if username in users:
    print("Username already exists")
    return

users[username] = {
    "name": name,
    "password": password
}

save_users(users)
```

```
    print("Signup Successful!")


# ========================================
# 5. USER LOGIN
# ========================================

def login(users):

    print("\n--- LOGIN ---")

    username = input(
        "Enter Username: "
    ).strip().lower()

    password = input("Enter Password: ")

    if username not in users:
        print("Invalid Username")
        return None

    if users[username]["password"] == password:
```

```
        print("Login Successful!")
        print("Welcome", users[username]["name"])

        return username

    print("Incorrect Password")

    return None


# ========================================
# 6. SUPABASE DATABASE CHAT HISTORY
# ========================================

class SupabaseChatMessageHistory(
    BaseChatMessageHistory
):

    def __init__(self, session_id):
        self.session_id = session_id
```

```
    @property
    def messages(self):

        response = (
            supabase
            .table(HISTORY_TABLE)
            .select("messages")
            .eq("session_id", self.session_id)
            .execute()
        )

        if not response.data:
            return []

        data = response.data[0]["messages"]

        if not data:
            return []

        return messages_from_dict(data)

    def add_messages(self, messages):
```

```
        all_messages = self.messages + messages

        data = [
            message_to_dict(message)
            for message in all_messages
        ]

        (
            supabase
            .table(HISTORY_TABLE)
            .upsert(
                {
                    "session_id": self.session_id,
                    "messages": data
                },
                on_conflict="session_id"
            )
            .execute()
        )

    def clear(self):
```

```
        (
            supabase
            .table(HISTORY_TABLE)
            .upsert(
                {
                    "session_id": self.session_id,
                    "messages": []
                },
                on_conflict="session_id"
            )
            .execute()
        )


# ========================================
# 7. CREATE OPENAI MODEL
# ========================================

llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.environ["OPENAI_API_KEY"]
```

```
)


# ========================================
# 8. CREATE PROMPT
# ========================================

prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a helpful AI assistant. "
            "Answer in simple English. "
            "The user's name is {name}."
        ),

        MessagesPlaceholder("history"),

        ("human", "{question}")
    ]
)
```

```
# ========================================
# 9. CREATE LCEL CHAIN
# ========================================

chain = prompt | llm


# ========================================
# 10. GET SESSION HISTORY
# ========================================

def get_session_history(session_id):

    return SupabaseChatMessageHistory(
        session_id
    )


# ========================================
# 11. CREATE CHATBOT WITH HISTORY
# ========================================
```

```
chatbot = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="question",
    history_messages_key="history"
)


# ========================================
# 12. LOAD REGISTERED USERS
# ========================================

users = load_users()

current_user = None


# ========================================
# 13. START APPLICATION
# ========================================
```

```
print("\nMULTI-USER AI CHATBOT")
print("SUPABASE JSONB DATABASE")


while True:

    if current_user is None:

        print("\n1. Signup")
        print("2. Login")
        print("3. Exit")

        choice = input("Enter Choice: ")

        if choice == "1":
            signup(users)

        elif choice == "2":
            current_user = login(users)

        elif choice == "3":
            break
```

```
        else:
            print("Invalid Choice")

    else:

        question = input(
            f"\n{current_user}: "
        ).strip()

        if question.lower() == "logout":

            print("Logged Out")
            current_user = None

        elif question.lower() == "exit":

            break

        elif question:

            name = users[current_user]["name"]
```

```
try:

    response = chatbot.invoke(
        {
            "name": name,
            "question": question
        },
        config={
            "configurable": {
                "session_id": current_user
            }
        }
    )

    print("\nAI:", response.content)

except Exception as error:

    print("Error:", error)
```

```
print("\nProgram Ended")
```

23. Final Memory Line

```
                USER LOGIN
                    ↓
          USERNAME = SESSION ID
                    ↓
     LOAD JSONB HISTORY FROM SUPABASE
                    ↓
       PROMPT + HISTORY + QUESTION
                    ↓
              LLM RESPONSE
                    ↓
       SAVE UPDATED JSONB HISTORY
                    ↓
     NEXT QUESTION / NEXT PROGRAM RUN
```

One-Line Conclusion

This project uses Supabase PostgreSQL JSONB as persistent cloud storage so every

logged-in user can maintain a separate LangChain conversation history even after the Python program stops and restarts.